Whole-Line Column Generation for Multi-Line Human–Robot Collaborative Selective Disassembly With Cumulative Fatigue

Author Names and Affiliations to Be Confirmed

***Abstract*—Multi-line disassembly requires coordination of limited human and robot resources for profitable material recovery. This work considers selective task execution, three operation modes, AND precedence relationships, and cumulative station fatigue. A mixed-integer linear model maximizes net profit under cycle-time, recovery, and shared-resource constraints. A whole-line column generation framework integrates feasible line plans, two-phase resource coordination, restricted integer recovery, and demand-driven resource-profile bounds. Computational experiments establish integer optimality for six synthetic instances with 80 or 120 tasks in 19.36–182.69 seconds under a 600-second protocol, whereas direct compact-model optimization remains unproven on these instances; additional resource-constrained cases confirm the benefit, and a difficult two-line case identifies its boundary.**

***Index Terms*—Human–robot collaboration, disassembly line balancing, column generation, cumulative fatigue, resource allocation.**

I. Introduction

DISASSEMBLY supports the recovery of reusable components and materials from end-of-life products. A disassembly line balancing problem (DLBP) assigns selected tasks to ordered workstations. Task selection affects recovery value. Workstation allocation affects time, resource demand, and operating cost. Guo *et al.* describe the modeling and optimization challenges in disassembly sequence planning [1].

Human–robot collaboration provides several task execution options. A human can perform a manual task, a robot can perform an automated task, and a joint mode can use both resources. Guo *et al.* study stochastic operation time with a multi-objective shuffled frog leaping method [2]. Guo *et al.* also examine partial destructive disassembly with human–robot hybrid operation [3]. Chu and Chen examine selective operations in a two-sided cooperative line [4]. Huang *et al.* consider uncertain operation and equipment activation in a U-shaped line [5]. These studies motivate explicit execution-mode and resource decisions. Their layouts and objectives differ from the multi-line setting considered here.

Coordination becomes important when several lines draw resources from common pools. Local plans can satisfy every task and station requirement, yet their combined staffing demand can exceed the available resources. Guo *et al.* model multiproduct collaborative hybrid lines with resource sharing and solve them with a discrete aquila optimizer [6]. Gu *et al.* address shared transportation in multi-factory remanufacturing [7]. Zhou *et al.* consider worker fatigue in multi-factory process optimization [8]. These studies establish the relevance of resource coordination and human factors. The present work focuses on fixed human and robot deployment across disassembly lines. It uses a station-level fatigue limit over one representative cycle. Its computational contribution concerns line-plan decomposition and auditable integer bounds, rather than the general introduction of resource sharing.

A compact mixed-integer linear programming (MILP) formulation provides a direct optimization model. Its relaxation can mix fractional task assignments within stations. An alternative formulation represents an entire feasible line plan by one column. All line-specific constraints then remain inside the pricing domain. Only two resource capacities connect the lines. Dantzig and Wolfe establish the decomposition principle for block-structured models [9]. Lübbecke and Desrosiers explain the role of pricing and dual bounds in column generation (CG) [10]. This theory motivates the present formulation; decomposition itself is an established method.

The computational question is whether line-level convexification reduces the cost of obtaining profitable, certifiable integer plans. A stronger relaxation alone does not answer this question. Each pricing problem remains an integer optimization problem. Large line blocks can make pricing expensive. Fractional master solutions can also leave an integer gap. A practical framework therefore needs both line pricing and a separate mechanism for integer solutions and bounds.

This work makes three contributions. First, it formulates a profit-maximizing multi-line DLBP with selective tasks, three execution modes, cumulative fatigue, and shared resource pools. Second, it develops a whole-line reformulation with an integer equivalence proof, two-phase CG, and demand-driven resource-profile coordination. Third, it evaluates the framework against direct compact-model optimization on frozen synthetic inputs. The evaluation reports valid bounds and proof states as well as execution time. It also identifies a failure regime with few, large pricing blocks.

Section II defines the problem and compact model. Section III presents the decomposition and integer coordination. Section IV reports the computational evidence. Section V states the conclusions and remaining questions.

II. Problem Description and Mathematical Model

A. System Definition and Assumptions

Each line has its own task set and an AND precedence graph. An executed successor requires every direct predecessor. Optional tasks can remain unexecuted. Required tasks

TABLE I
SETS AND PARAMETERS

Symbol	Definition
$\mathcal{L}$	Set of disassembly lines; index l.
$\mathcal{I}_l, \mathcal{I}_l^{\mathrm{req}}$	Tasks and required tasks of line l; index i.
$\mathcal{S}_l = \{1, \dots, k_l\}$	Ordered candidate stations; index s.
$\mathcal{M}_{li} \subseteq \{H, R, J\}$	Allowed execution modes; index m.
$\mathcal{E}_l$	Direct AND precedence arcs (i, j).
n_l	Number of tasks on line l.
v_{li}, w_{li}	Recovery value and recovery weight of task i.
$c_{li}^m, t_{li}^m, \beta_{li}^m$	Execution cost, duration, and fatigue coefficient.
$c_{ls}^0, c_{ls}^H, c_{ls}^R$	Activation, human, and robot costs.
$t_l^c, \bar{f}_l, b_l$	Cycle time, fatigue threshold, and fatigue budget.
q_l	Minimum recovery weight of line l.
$h^{\max}, r^{\max}$	Shared human and robot capacities.

must be executed. Each selected task receives one allowed mode and one station. Stations have ordered indices. A predecessor can occupy the same station as its successor. A topological order provides a feasible within-station sequence.

Modes H, R, and J denote human, robot, and joint execution. Mode J requires one human and one robot. Its duration enters station time once. Tasks within a station execute sequentially. Resource pools limit the total number of stations with human and robot deployment. This planning model does not describe resource movement between stations during operation.

Fatigue starts at zero in a representative cycle. Each human-involving task contributes a nonnegative additive load. The model contains no within-cycle recovery. It does not describe fatigue accumulation across successive cycles or an entire work shift. Robot-only modes have zero fatigue load. Candidate stations can have different costs. The formulation does not require consecutive station activation.

B. Compact Formulation

Table I defines the main notation. Binary y_{li} selects task i. Binary x_{lism} assigns task i to station s in mode m. Binary u_{ls}, h_{ls}, and r_{ls} indicate station activation, human deployment, and robot deployment. Let a_m^H and a_m^R be resource indicators. Their pairs are $(1,0)$, $(0,1)$, and $(1,1)$ for H, R, and J.

The objective maximizes net profit:

$$\max\ z = \sum_{l,i} v_{li} y_{li} - \sum_{l,i,s,m} c_{li}^m x_{lism} - \sum_{l,s} \left(c_{ls}^0 u_{ls} + c_{ls}^H h_{ls} + c_{ls}^R r_{ls} \right). \tag{1}$$

Every sum over m uses the allowed set $\mathcal{M}_{li}$. The assignment and required-task constraints are

$$\sum_{s \in \mathcal{S}_l} \sum_{m \in \mathcal{M}_{li}} x_{lism} = y_{li} \qquad \forall l, i, \tag{2}$$

$$y_{li} = 1 \qquad \forall l,\ i \in \mathcal{I}_l^{\mathrm{req}}. \tag{3}$$

AND precedence uses cumulative prefixes for all l, $(i,j) \in \mathcal{E}_l$, and $k \in \mathcal{S}_l$:

$$\sum_{s=1}^{k} \sum_{m \in \mathcal{M}_{lj}} x_{ljsm} \le \sum_{s=1}^{k} \sum_{m \in \mathcal{M}_{li}} x_{lism}. \tag{4}$$

The final prefix enforces predecessor selection. Earlier prefixes enforce station order. Each station satisfies

$$\sum_{i,m} t_{li}^m x_{lism} \le t_l^c u_{ls}, \tag{5}$$

$$u_{ls} \le \sum_{i,m} x_{lism} \le n_l u_{ls}, \tag{6}$$

$$h_{ls} \le \sum_{i,m} a_m^H x_{lism} \le n_l h_{ls}, \tag{7}$$

$$r_{ls} \le \sum_{i,m} a_m^R x_{lism} \le n_l r_{ls}, \tag{8}$$

$$\sum_{i,m} d_{li}^m x_{lism} \le b_l h_{ls}. \tag{9}$$

These constraints apply to all l, s. The fatigue load is $d_{li}^m = \beta_{li}^m t_{li}^m$. The budget is $b_l = -\ln(1 - \bar{f}_l)$. Thus the fatigue level $1 - \exp(-\sum_{i,m} d_{li}^m x_{lism})$ does not exceed $\bar{f}_l$. The deployment constraints make $h_{ls} = 0$ exactly when no selected mode needs a human. Equivalent logic holds for robots and station activation. An empty station has zero deployment and zero activation cost.

Shared resources and line recovery satisfy

$$\sum_{l,s} h_{ls} \le h^{\max}, \qquad \sum_{l,s} r_{ls} \le r^{\max}, \tag{10}$$

$$\sum_{i \in \mathcal{I}_l} w_{li} y_{li} \ge q_l \qquad \forall l. \tag{11}$$

All decision variables are binary. Equations (2)–(11) retain the task, precedence, mode, cycle-time, fatigue, and recovery definitions of the implemented compact model. There are no precedence arcs between different lines.

III. WHOLE-LINE DECOMPOSITION AND SOLUTION FRAMEWORK

A. Whole-Line Plans

Let $\mathcal{X}_l$ contain all binary solutions of line l that satisfy (2)–(9) and (11). The shared limits in (10) remain outside this set. A pricing model can use the redundant bounds $\sum_s h_{ls} \le k_l$ and $\sum_s r_{ls} \le k_l$. These bounds do not allocate global resources to a line.

A column $p \in \mathcal{P}_l$ represents one complete plan in $\mathcal{X}_l$. It records the selected tasks, their stations and modes, and the associated station configurations. Define

$$h_{lp} = \sum_s h_{ls}^p, \qquad r_{lp} = \sum_s r_{ls}^p, \tag{12}$$

$$c_{lp} = \sum_i v_{li} y_{li}^p - \sum_{i,s,m} c_{li}^m x_{lism}^p - \sum_s (c_{ls}^0 u_{ls}^p + c_{ls}^H h_{ls}^p + c_{ls}^R r_{ls}^p). \tag{13}$$

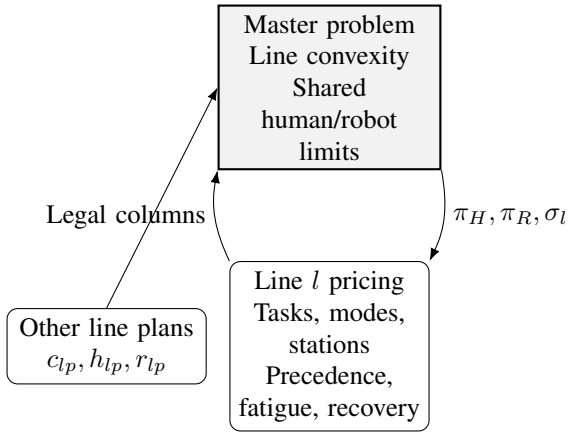


Fig. 1. Whole-line coordination. Each pricing domain contains all local task and station constraints. Only human and robot capacities connect the lines.

An empty station is allowed within a plan. A completely empty line plan exists only if the required-task and recovery conditions allow it. Columns with the same selected task set can still differ in station assignment or mode. Deduplication therefore uses the complete assignment pattern.

Fig. 1 shows the block structure. A station-level column can be locally feasible but difficult to combine across stations. A whole-line column already satisfies task uniqueness, required-task selection, all precedence prefixes, and line recovery. Integer combination then requires coordination of the two resource totals.

B. Master Problem and Formulation Strength

Binary λ_{lp} selects one plan for line l. The complete master problem is

$$\max \quad \sum_{l} \sum_{p \in \mathcal{P}_l} c_{lp} \lambda_{lp}, \tag{14}$$

$$\text{s.t.} \quad \sum_{p \in \mathcal{P}_l} \lambda_{lp} = 1 \qquad \forall l, \tag{15}$$

$$\sum_{l,p} h_{lp} \lambda_{lp} \le h^{\max}, \tag{16}$$

$$\sum_{l,p} r_{lp} \lambda_{lp} \le r^{\max}, \tag{17}$$

$$\lambda_{lp} \in \{0,1\}. \tag{18}$$

Column profit already includes all station and resource costs.

Proposition 1. *The complete integer master and the compact MILP have the same feasible objective values.*

Proof. Extract the binary variables of each line from a feasible compact solution. They form a member of $\mathcal{X}_l$ and hence a column. Selecting these columns preserves the resource totals and every objective term. Conversely, select one column per line from a feasible integer master solution. Copy its task, station, and mode variables to the compact model. Local feasibility follows from $\mathcal{X}_l$. Shared feasibility follows from (16) and (17). The two-way configuration logic fixes activation and deployment, including empty stations. The reconstructed profit is unchanged. □

The complete linear programming (LP) relaxation replaces (18) by $\lambda_{lp} \ge 0$. Let $z^{\mathrm{LP}}_{\mathrm{line}}$, $z^{\mathrm{LP}}_{\mathrm{station}}$, and $z^{\mathrm{LP}}_{\mathrm{compact}}$ denote the complete line, complete station, and compact LP bounds. Then

$$z^{\mathrm{IP}} \le z^{\mathrm{LP}}_{\mathrm{line}} \le z^{\mathrm{LP}}_{\mathrm{station}} \le z^{\mathrm{LP}}_{\mathrm{compact}}. \tag{19}$$

Every line-plan mixture induces station-plan mixtures with identical weights and objective. Station-plan mixtures in turn induce feasible compact fractional assignments. This establishes the inclusions in (19). The supplementary file gives the detailed mapping. These comparisons concern complete column sets, not arbitrary restricted pools.

C. Restricted Master and Exact Pricing

CG maintains a finite pool $\overline{\mathcal{P}}_l \subseteq \mathcal{P}_l$. Its restricted master problem (RMP) uses the LP version of (14)–(17) over this pool. The dual variables are $\sigma_l \in \mathbb{R}$ for (15) and $\pi_H, \pi_R \ge 0$ for the resource rows. The complete dual is

$$\min \quad D = \sum_{l} \sigma_l + h^{\max} \pi_H + r^{\max} \pi_R, \tag{20}$$

$$\text{s.t.} \quad \sigma_l + \pi_H h_{lp} + \pi_R r_{lp} \ge c_{lp} \qquad \forall l, p. \tag{21}$$

The reduced cost of a column is

$$\bar{c}_{lp} = c_{lp} - \pi_H h_{lp} - \pi_R r_{lp} - \sigma_l. \tag{22}$$

Pricing solves one MILP per line:

$$\rho_l = \max_{p \in \mathcal{P}_l} \{ c_{lp} - \pi_H h_{lp} - \pi_R r_{lp} - \sigma_l \}. \tag{23}$$

The objective increases the effective deployment costs by π_H and π_R. All line constraints remain unchanged. Task coverage and precedence do not have separate master duals in this formulation. They belong to the pricing domain.

A new legal column enters the pool if its independently recomputed reduced cost exceeds ε_{rc}. All lines use the same RMP dual solution in a round. The implementation reuses one native CPLEX model per line. It updates every objective coefficient and submits the previous full integer solution as a warm start. Separate line sessions prevent state contamination.

D. Initialization and Phase I

A short economic MILP first seeks a feasible plan on each line. A constant-objective solve provides a fallback. A proven empty local domain implies global infeasibility. A time limit without a plan does not establish emptiness.

Local plans can conflict in their combined resource demands. Phase I keeps each convexity equation hard and adds nonnegative elastic variables:

$$\max \quad -a_H - a_R, \tag{24}$$

$$\text{s.t.} \quad \sum_{p} \lambda_{lp} = 1 \qquad \forall l,$$

$$\sum_{l,p} h_{lp} \lambda_{lp} - a_H \le h^{\max},$$

$$\sum_{l,p} r_{lp} \lambda_{lp} - a_R \le r^{\max},$$

$$\lambda_{lp}, a_H, a_R \ge 0.$$

One local plan per line makes this elastic RMP feasible. Real columns have zero objective coefficients. The dual satisfies $0 \le \pi_H, \pi_R \le 1$. Phase I pricing maximizes $-\pi_H h_{lp} - \pi_R r_{lp} - \sigma_l$ over $\mathcal{P}_l$.

The complete Phase I optimum is zero exactly when the complete line LP is feasible. A feasible line LP admits zero artificials. Conversely, zero artificials leave all real master constraints satisfied. This equivalence concerns LP feasibility. It does not establish integer feasibility. After the artificial sum reaches its tolerance, the implementation resolves the hard Phase II RMP before proceeding.

E. Valid Bounds and Integer Recovery

At an optimal RMP solution, repair each convexity dual by $\sigma'_l = \sigma_l + \max(0, \rho_l)$. Complete exact pricing then gives

$$z^{\mathrm{LP}}_{\mathrm{line}} \le u^{CG} = D + \sum_l \max(0, \rho_l). \tag{25}$$

The repaired variables satisfy every complete-dual inequality. This proves the bound by weak duality. When every $\rho_l \le \varepsilon_{rc}$, the residual LP gap can accumulate across the lines. It is not automatically bounded by one pricing tolerance.

An incomplete or time-limited pricing round does not certify (25). The implementation still attempts the remaining lines under the same duals. It then stops that CG loop if any pricing maximum remains unproven. Existing valid bounds remain available. A negative complete Phase I repaired bound proves LP infeasibility.

The restricted integer master provides candidate integer plans. A data-level checker and the original compact constraints verify each plan before its profit becomes a lower bound (LB). Restricted integer optimality alone does not imply global integer optimality. Independently, local economic upper bounds $b^e_l \ge \max_{p \in \mathcal{P}_l} c_{lp}$ give $z^{\mathrm{IP}} \le z^{\mathrm{LP}}_{\mathrm{line}} \le \sum_l b^e_l$. The implementation also reuses reliable native bounds from full-domain, zero-resource-price economic pricing. It restores the convexity constant before using these bounds. It does not substitute an incumbent for a maximization upper bound (UB).

F. Demand-Driven Resource Profiles

When the current LB and UB remain separated, define

$$v_l(a, b) = \max\{c_{lp} : p \in \mathcal{P}_l,\ h_{lp} = a,\ r_{lp} = b\}. \tag{26}$$

The integer counts satisfy $a \le \min(k_l, h^{\max})$ and $b \le \min(k_l, r^{\max})$. An infeasible profile has value $-\infty$. A profile with no completed solve remains unresolved. Plans with identical resource totals have identical coupling coefficients. Only the most profitable plan of such a profile is needed for integer coordination.

Let $\underline{v}_l$ contain validated profile incumbents and $\overline{v}_l$ contain valid profile upper bounds. Unknown upper bounds remain $+\infty$. Prior complete pricing gives

$$\overline{v}_l(a, b) \le \sigma_l + \max(0, \rho_l) + \pi_H a + \pi_R b. \tag{27}$$

The right-hand side is a valid candidate bound; the stored value takes the minimum over valid sources. For a profile table v, dynamic programming (DP) uses

$$f_l(A, B) = \max_{a \le A, b \le B} \{f_{l-1}(A - a, B - b) + v_l(a, b)\}, \tag{28}$$

with $f_0(A, B) = 0$. The lower table yields a feasible integer combination. The upper table bounds every feasible integer combination. Its bound can be tighter than the complete line LP bound. Closing this integer bound does not certify LP convergence.

The procedure queries an unresolved profile on an upper-DP maximizing path. It chooses the least-attempted profile and uses deterministic line and profile ordering to break ties. A query fixes exact resource counts in a local economic MILP. Its incumbent improves the lower table. Its valid native bound improves the upper table. Its proven infeasibility removes that profile. Query budgets increase geometrically with repeated attempts. The procedure recomputes both DP tables until the budget ends or the bounds close.

Algorithm 1 Whole-Line CG With Integer Coordination

Input: Line data, shared capacities, time budget, tolerances.
Output: Verified plan, bounds, and proof state.

```
Generate one feasible plan per line by local MILP.
Record reliable local economic bounds.
Check independent-plan compatibility and bound closure.
Solve elastic Phase I by CG; retain explicit pricing states.
Resolve the hard Phase II RMP after zero artificials.
while time remains and Phase II remains unresolved do
    Solve the RMP and verify its primal and dual solution.
    Recover and verify a restricted integer solution.
    Stop if the verified LB closes the valid UB.
    Price every line under the current duals.
    Add legal new positive-reduced-cost columns.
    Update the UB only after complete exact pricing.
    Stop CG on an incomplete pricing proof or LP convergence.
end while
Query unresolved resource profiles through upper-DP paths.
Update integer bounds and verify the selected plan.
Return the actual bound, proof state, and termination reason.
```

Algorithm 1 summarizes the implemented framework. Let the best verified integer profit be ℓ and the best valid integer upper bound be u. Define

$$g_{abs} = u - \ell, \qquad g_{rel} = \frac{\max(0, u - \ell)}{\max(1, |\ell|)}. \tag{29}$$

Missing bounds produce an unknown gap. Integer optimality requires

$$u - \ell \le \max\{\varepsilon_{abs}, \varepsilon_{rel} \max(1, |\ell|)\}. \tag{30}$$

A bound below the verified LB beyond the consistency tolerance triggers a numerical error. All proofs use solver floating-point contracts. They are not exact rational certificates. The reported implementation combines root CG and profile DP. Its experiments do not measure a branch-and-price search tree.

IV. Experimental Studies

A. Test Instances and Evaluation Protocol

The experiments use synthetic inputs from a seeded generator. They do not represent measured industrial products. The main set contains three seeds, 32–34, at two configurations: 80 tasks on four lines and 120 tasks on six lines. Each line has 20 tasks and eight candidate stations. Both methods read the same frozen input bytes. The task graph is layered and contains only within-line forward arcs. Required tasks occupy the first 20% of each line's task list. Recovery requires at least 35% of the total task weight, rounded upward.

The generator samples mode availability and task economics before partitioning the task bank. It provides a structurally feasible construction for input checks. Neither solver receives this construction as an initial solution. Shared capacities in the main set equal the observed independent economic calibration counts minus one for each resource. This calibration defines a development design, not a random industrial population. The supplementary file provides the generator rules and all capacity values.

A second set retains the 120-task inputs and sets each shared capacity to the ceiling of 80% of its calibrated independent use. A retained 80-task two-line input provides a large-block boundary. A separate structure study compares two, four, and eight lines at 80 total tasks. Repartitioning retains the task bank but changes precedence graphs and recovery rows. These are different mathematical inputs. The study therefore does not isolate line count as the only causal factor.

The direct baseline solves the compact MILP with CPLEX. The decomposition also uses CPLEX inside line pricing, profile queries, and integer recovery. All measured pairs use the same machine, CPLEX 22.1.1, one thread, random seed 11, and zero requested MIP gap. The archived environment contains Python 3.10.11, PuLP 3.3.2, and Windows 10. The physical processor model is not preserved in the archived environment and requires confirmation for submission.

The main protocol allocates 600 seconds per method. The resource-constrained protocol allocates 180 seconds. Both reserve five seconds for evidence finalization. A parent process measures execution from worker startup through exit, including required result persistence. Independent offline replay is separate. The watchdog and native solver limits operate under Windows scheduling; the protocol is not a hard real-time guarantee. All listed main and robustness workers return timely, complete evidence. A time-limited CPLEX completion time is censored. It is not an observed time to optimality. No 3600-second run enters these tables.

B. Parameters and Performance Metrics

The framework uses a 0.5-second economic initialization per line and a 0.5-second feasibility fallback when necessary. Restricted integer recovery receives up to 0.5 seconds per call. CG receives at most 90% of the available internal budget. The pricing loop divides remaining time among the remaining lines. Initial profile queries receive up to two seconds. Repeated attempts double that allowance, subject to the remaining global time. The maximum CG record count is 100. These settings are fixed for the reported main and resource-constrained sets. They are implementation settings, not parameters inferred from a cited study.

The LP primal and dual tolerances are 10^{-8}. Pricing integrality, reduced-cost, and artificial tolerances are 10^{-7}. Original-solution verification uses 10^{-6}. Bound consistency and absolute integer closure use 10^{-5}. Relative integer closure uses 10^{-8}. The supplementary file lists all numerical roles.

The primary metrics are the validated integer LB, valid UB, gap in (29), integer proof state, and measured execution time. Additional metrics include pricing calls, generated columns, Phase I and II pricing rounds, and native solver time. A round count excludes RMP-only phase transitions and early closure records. Compact-model search nodes and CG rounds represent different operations and are not directly compared.

TABLE II
Small Complete-Column Checks

Case	Line LP	Full integer	Profile result
Five-task reference	24.00	24.00	Optimal
Six-task reference	56.63	56.63	Optimal
Positive-gap case	6.50	2.00	Optimal
LP-feasible/IP-infeasible	4.00	Infeasible	Infeasible

C. Feasibility and Integer-Certificate Checks

Small instances use an independent enumerator of task omission, station assignment, and allowed mode. The enumerator checks task selection, AND precedence, time, fatigue, deployment, and recovery directly from data. An independently assembled complete-column LP and integer model provide reference values. Archived checks also compare CPLEX and CBC on these small cases.

Table II confirms the need to distinguish LP and integer conclusions. The positive-gap case requires integer coordination despite exact CG convergence. The LP-feasible/IP-infeasible case contains no integer witness. Resource-profile analysis proves its integer infeasibility. These small archived tests establish implementation consistency within their enumerated scope. They do not replace larger-instance performance evaluation.

D. Main Comparison With Compact CPLEX

Table III shows an integer-certification advantage on all six main inputs. The framework establishes optimality in 19.36–182.69 seconds. Compact CPLEX remains unproven within the 600-second protocol. On the three 80-task inputs, both methods reach the same final integer profit. Their difference concerns proof completion. On two 120-task inputs, the framework also returns a higher final integer profit. On seed 34, Compact CPLEX reaches the same profit but retains a positive bound gap.

Fig. 2 presents the same comparisons without converting cutoffs into exact speedup ratios. These results support efficient certification on the tested multi-line configurations. They

TABLE III
PROFIT AND BOUNDS UNDER THE 600-SECOND PROTOCOL

Instance N/L/seed	Compact CPLEX				Whole-line framework		
	LB	UB	Gap (%)	Time (s)	LB	UB	Time (s)
80/4/32	583.64	618.26	5.93	595.68	583.64	583.64	19.36
80/4/33	675.21	719.63	6.58	595.73	675.21	675.21	24.60
80/4/34	667.64	720.51	7.92	595.84	667.64	667.64	68.68
120/6/32	954.82	1007.66	5.53	595.75	963.44	963.44	36.51
120/6/33	1009.46	1066.51	5.65	595.78	1012.19	1012.19	105.57
120/6/34	1041.21	1097.26	5.38	595.80	1041.21	1041.21	182.69

The whole-line framework proves integer optimality in every listed row. Compact CPLEX remains time-limited and unproven. Times include required evidence persistence. Rounded equal bounds indicate closure within the recorded tolerances.

TABLE IV
RESOURCE-CONSTRAINED AND LARGE-BLOCK CASES UNDER THE 180-SECOND PROTOCOL

Instance N/L/seed	$h^{\max}/r^{\max}$	Compact CPLEX			Whole-line framework			
		LB	UB	Time (s)	LB	UB	Time (s)	Proven
80/2/31	6/7	693.08	733.55	175.64	569.53	738.73	174.97	No
120/6/32	10/9	911.91	1010.76	175.59	927.46	927.46	30.73	Yes
120/6/33	9/10	955.90	1069.88	175.56	971.93	971.93	61.51	Yes
120/6/34	10/10	980.67	1101.05	175.55	1009.98	1009.98	144.55	Yes

The 120-task inputs use 80% of calibrated independent resource use, rounded upward. The 80-task two-line row is the retained large-block boundary. Compact CPLEX is unproven in all four rows.

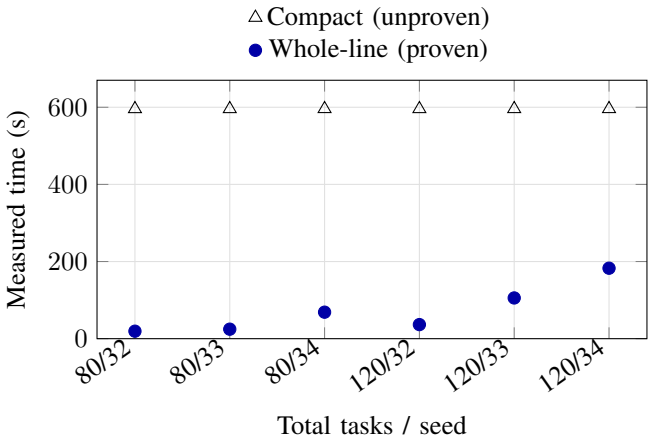


Fig. 2. Measured times for the main inputs, ordered by total tasks and seed. Open triangles indicate time-limited, unproven compact results. Filled circles indicate completed integer proofs. Compact markers are censored observations, not times to optimality.

do not establish that the framework first discovers an optimal-profit plan earlier. Comparable first-optimum timestamps are not available for Compact CPLEX.

E. Resource Coordination and a Large-Block Boundary

Table IV extends the comparison to stronger resource competition. The framework proves the three 120-task optima under the 180-second protocol. Compact CPLEX remains unproven and returns lower profits. Seeds 32 and 33 close through verified integer plans and complete CG bounds. Seed 34 requires 31 demand-driven profile queries after an incomplete CG pricing round. Its final certificate concerns the original integer optimum. It does not assert complete line-LP convergence.

The two-line case in Table IV provides a contrary result. Two lines with 40 tasks each produce difficult local MILPs. The decomposition returns both a lower profit and a looser UB than Compact CPLEX. Its CG pricing consumes about 156 seconds, and subsequent profile queries do not close the gap. Thus total task count alone does not predict the benefit of decomposition.

The separate 80-task structure study provides further context. A resource-constrained eight-line input reaches the same proven optimum under both methods. The framework completes in about 2.91 seconds; Compact CPLEX completes in about 128.09 seconds. A resource-ample eight-line input instead favors Compact CPLEX, at about 1.24 versus 1.64 seconds. The supplementary tables retain every compact/decomposition pair in that structure study. The evidence supports a tradeoff between block difficulty and coordination, rather than a universal benefit from adding lines.

F. Iterations, Cost, and Interpretation

Table V shows that Phase II needs only three to five pricing rounds in the main experiment. Native pricing nevertheless accounts for about 91–99% of total framework time. Each RMP uses less than 0.01 second of native solve time. The restricted integer master also remains small. These observations identify local pricing difficulty as the main cost in this regime.

The whole-line formulation can strengthen the relaxation through local integer convexification. It can also simplify integer combination by placing all within-line compatibility inside each column. The experiments are consistent with these mechanisms. They do not isolate relaxation strength

TABLE V
PRICING ROUNDS AND NATIVE SOLVER COSTS IN THE MAIN EXPERIMENT

N/L/seed	i_I	i_{II}	PP	Cols.	PP (s)	RMP (s)
80/4/32	1	3	20	18	17.67	0.0064
80/4/33	1	4	24	22	22.79	0.0077
80/4/34	1	4	24	21	66.70	0.0073
120/6/32	1	3	30	26	34.24	0.0058
120/6/33	1	5	42	28	102.94	0.0076
120/6/34	1	4	36	27	180.31	0.0056

A pricing round includes an RMP solution and at least one pricing attempt under its duals. PP includes economic initialization. RMP-only transition and bound-closure records do not count as pricing rounds. Resource-profile query counts are zero in these six runs.

from warm-start behavior or solver search-state effects. A full component ablation is still required to quantify each contribution.

The evaluation has a defined boundary. It uses a small synthetic development suite and one solver seed per measured input. Resource calibration is part of the instance construction. The suite does not establish statistical significance, industrial ergonomic validity, or performance on arbitrary large line blocks. Broader comparator algorithms, measured product cases, and an independent final test suite remain necessary before a broad state-of-the-art claim.

V. CONCLUSION

This work develops a whole-line decomposition framework for multi-line selective human–robot disassembly with cumulative fatigue. Complete line plans preserve local feasibility and expose only two shared-resource totals to the master problem. Two-phase CG supplies LP coordination and valid bounds. Restricted integer recovery and demand-driven resource profiles supply separate integer witnesses and proof paths.

The tested multi-line configurations show a certification advantage over direct compact-model optimization. The retained large-block case defines a practical limitation: difficult local MILPs can dominate the computation. The results therefore support decomposition for the tested structure, rather than unconditional superiority at larger task counts.

Further investigation concerns stronger line-pricing bounds, allocation of certification budgets, and measured-product validation. Learning-assisted control is a possible extension for candidate generation and certification decisions. The present results contain no trained learning policy or evidence of a reinforcement-learning speedup.

DATA AND CODE AVAILABILITY

The implementation, synthetic inputs, and archived numerical certificates are available at https://github.com/ZzzHry/firstEssay. The supplementary file provides the artifact paths and verification scope. No industrial measurement data enter the reported experiments.

AUTHOR INFORMATION AND DECLARATIONS

Author identities, affiliations, contributions, funding, and competing-interest declarations require confirmation before submission. Author photographs and biographies are pending.

REFERENCES

[1] X. Guo, M. Zhou, A. Abusorrah, F. Alsokhiry, and K. Sedraoui, "Disassembly sequence planning: A survey," *IEEE/CAA Journal of Automatica Sinica*, vol. 8, no. 7, pp. 1308–1324, 2021.

[2] X. Guo, C. Fan, M. Zhou, S. Liu, J. Wang, S. Qin, and Y. Tang, "Human–robot collaborative disassembly line balancing problem with stochastic operation time and a solution via multi-objective shuffled frog leaping algorithm," *IEEE Transactions on Automation Science and Engineering*, vol. 21, no. 3, pp. 4448–4459, 2024.

[3] L. Guo, Z. Zhang, T. Wu, Y. Zhang, Y. Zeng, and X. Xie, "Green and efficient-oriented human-robot hybrid partial destructive disassembly line balancing problem from non-disassemblability of components and noise pollution," *Robotics and Computer-Integrated Manufacturing*, vol. 90, p. 102816, 2024.

[4] M. Chu and W. Chen, "Human-robot cooperation two-sided partial disassembly line balancing problem," *International Journal of Production Research*, vol. 63, no. 12, pp. 4478–4503, 2025.

[5] Z. Huang, H. Zhang, G. Tian, M. Yang, D. Wang, and Z. Li, "Energy-efficient human-robot collaborative U-shaped disassembly line balancing problem considering turn on-off strategy: Uncertain modeling and solution method," *Journal of Manufacturing Systems*, vol. 80, pp. 38–69, 2025.

[6] X. Guo, F. Guo, L. Qi, J. Wang, S. Liu, S. Qin, and W. Wang, "Modeling and optimization of multiproduct human–robot collaborative hybrid disassembly line balancing with resource sharing," *IEEE Transactions on Computational Social Systems*, vol. 12, no. 5, pp. 2848–2863, 2025.

[7] J. Gu, Z. Guo, J. Wang, L. Qi, S. Qin, and S. Zhang, "Optimization of multi-factory remanufacturing processes with shared transportation resources using the ALNS algorithm," *International Journal of Artificial Intelligence and Green Manufacturing*, vol. 1, no. 1, pp. 1–13, 2025.

[8] L. Zhou, H. Zhu, and B. Akbari, "Multi-objective optimization of multi-factory remanufacturing process considering worker fatigue," *International Journal of Artificial Intelligence and Green Manufacturing*, vol. 1, no. 2, pp. 93–107, 2025.

[9] G. B. Dantzig and P. Wolfe, "Decomposition principle for linear programs," *Operations Research*, vol. 8, no. 1, pp. 101–111, 1960.

[10] M. E. Lübbecke and J. Desrosiers, "Selected topics in column generation," *Operations Research*, vol. 53, no. 6, pp. 1007–1023, 2005.